Programación declarativa

• Haskell •

Doble grado en Ingeniería Informática y Matemáticas
Universidad Complutense de Madrid
Óscar Martín — Curso 2026/27

27. Compilación

`main`

- Un programa Haskell debe definir `main` para generar un ejecutable.

```
main :: IO ()
main = putStrLn "¡Hola, mundo!"
```

- El tipo de `main` puede ser `IO a` para cualquier `a`.
  - Pero el valor final se descarta.
- `module Main (main) where` se toma por defecto.
- El archivo que contiene este módulo puede tener cualquier nombre.
- Cada programa Haskell tiene una parte *pura* y una *impura*.
  - `IO` no aparece en la parte pura.
  - La parte impura debe ser lo más breve posible.

Compilación

- `ghc helloworld.hs`: compila y genera un ejecutable.
- `runhaskell helloworld.hs`: ejecuta sin generar un ejecutable (como `ghci`).
  - `runghc helloworld.hs` es lo mismo.
- El código compilado es mucho más rápido que el ejecutado en `ghci`.
- *Ojo*: En un programa compilado, la salida estándar va por líneas.
  - `putStr` sin `\n` puede no mostrarse hasta más tarde.
  - Solución:
    - con `putStrLn` o `print` no pasa;
    - `hFlush stdout` tras el `putStr` (están en `System.IO`).

Un programa completo

```haskell
import Data.Char (toUpper)

main :: IO ()
main = do putStrLn "1. Pasar a mayúsculas todo el archivo."
          putStrLn "2. Darle la vuelta al archivo."
          putStrLn "3. Terminar."
          opcion <- leeInt
          case opcion of
              1 -> do transformaArch aMayusculas
                      main
              2 -> do transformaArch alReves
                      main
              _ -> putStrLn "Hasta luego."

leeInt :: IO Int
leeInt = do linea <- getLine
            return (read linea)

transformaArch :: (String -> String) -> IO ()
transformaArch f = do
    putStrLn "Nombre del archivo:"
    nombre <- getLine
    contenido <- readFile nombre
    writeFile (nombre ++ ".nuevo") (f contenido)

-- ↓ parte pura ↓

aMayusculas :: String -> String
aMayusculas = map toUpper

alReves :: String -> String
alReves = unlines . reverse . lines
```

`interact`

```haskell
interact :: (String -> String) -> IO ()
interact f = do s <- getContents
                putStr (f s)
```

- La parte impura queda reducida a una línea:

```haskell
main = interact aMayusculas
```

- Es perezosa, como `readFile` y `getContents`: escribe a medida que lee.

Ejercicios

1. Escribe un programa que lea notas, una por línea, hasta que lea una línea vacía, y que después escriba un informe:

   ```
   5
   8
   3.5

   Notas: 3
   Media: 5.5
   Aprobados: 2
   Nota máxima: 8.0
   ```

   - Separa claramente la parte impura, `leeNotas :: IO [Double]`, de la pura, `informe :: [Double] -> String`.
     - Necesitarás guardar en una lista las notas que vayas leyendo.
   - Compílalo con `ghc` y ejecútalo.

2. Escribe con `interact` un programa que, para cada línea que escriba el usuario, escriba cuántas palabras tiene. Ejecútalo con `runghc`. Fíjate en el funcionamiento perezoso de `interact`.